

CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion

1 Paper summary

This paper looks at reducing the prefill cost in LLM inference by reusing KV caches when the same text appears across different requests. The main issue is that prefill can be quite expensive, especially when inputs include multiple chunks (like in retrieval-based setups).

Existing approaches have some limitations. Prefix caching works well but only if the reused text is at the beginning, while full KV reuse can handle more general cases but often hurts quality because it ignores cross-attention between chunks.

The paper proposes CacheBlend, which tries to combine the benefits of both. The idea is to reuse KV caches from different chunks, but also selectively recompute some of the KV states to recover the missing attention information. Interestingly, they show that usually only a small fraction of tokens (often less than 15%) needs to be recomputed, which makes the approach efficient.

To avoid adding extra latency, the system pipelines recomputation with loading KV caches for the next layer. This makes the overhead less visible, even if it adds complexity. It also allows storing KV caches on slower storage instead of only in fast memory.

The results show improvements over prefix caching, both in time-to-first-token and throughput, while maintaining similar output quality. So the main idea is to get the benefits of reuse without sacrificing too much accuracy.

2 Contributions

The first contribution is identifying the limitations of existing KV reuse methods. The paper shows that prefix caching is too restrictive, while full reuse can hurt quality, which highlights a gap in current systems.

The second contribution is the idea of selective KV recomputation. Instead of fully recomputing or fully reusing, the system mixes both approaches. This is important because it tries to balance performance and quality, even if the exact selection process is not trivial.

The third contribution is the pipelined implementation. By overlapping recomputation with cache loading, the system hides most of the extra cost. This makes the approach more practical, otherwise the added work could cancel out the benefits.

3 Problems

One limitation is that the approach depends on the assumption that only a small number of tokens need recomputation. This seems to hold in their experiments, but might not always be true for all workloads or models. If more tokens need updating, the performance gain could be reduced.

Another issue is the added complexity. The system has to manage multiple KV caches, decide what to recompute, and pipeline everything correctly. This makes it harder to implement and debug compared to simpler methods.

A third point is that the benefits depend on having reusable text across requests. In cases where inputs are very different or reuse is limited, the advantage of CacheBlend might be smaller. So it works best in specific scenarios, like retrieval-based applications.